dispatcher to an actor, a specific group of actors, or all actors in the system.

So when you send a message to an actor, you're really leaving a message behind in its mailbox. Eventually, a dispatcher will push the message through the actor. The actor, in turn, can leave a message behind for the next actor, and that message will be pushed through at some point.

Actors are lightweight because they run on top of dispatchers; the number of actors isn't necessarily directly proportional to the number of threads. Akka actors take a lot less space than threads: around 2.7 million actors can fit in 1 GB of memory. That's a big difference compared to 4096 threads for 1 GB of memory, so you can create different types of actors more freely than you could using threads directly.

You can choose different types of dispatchers that can be tuned to specific needs. Being able to configure and tune the dispatchers and mailboxes used throughout the application gives you a lot of flexibility when performance tuning.

Callback hell

A lot of frameworks provide asynchronous programming through callbacks. If you've used any of these frameworks, chances are good that you've been to a place called *callback hell*, where every callback calls another callback, which calls another callback, and so on.

Compare this approach to how the dispatcher chops up the messages in the mailbox and pushes them through on a given thread. Actors don't need to provide a callback in a callback, all the way down to some sulfur pit, which is good news. Actors drop off messages in a mailbox and let the dispatcher sort out the rest.